

ARDUINO SENSOR REPORT

IR Sensor • Ultrasonic Sensor • Temperature Sensor

Devaprasath S - 23BTT009

Nandiesh R V - 23BTT026

Narren Karthikeyan U - 23BTT027

Nikitha Y - 23BTT028

Pavishna Dhaniya R - 23BTT030

Rajesh M - 23BTT308

1. Introduction

Arduino is an open-source electronics platform that combines easy-to-use hardware and software, making it ideal for students, hobbyists, and professionals alike. At its core, the Arduino UNO is a microcontroller board based on the ATmega328P, featuring 14 digital I/O pins, 6 analog input pins, and a 16 MHz crystal oscillator. It serves as the brain behind countless IoT and embedded systems projects worldwide.

Sensors are the fundamental building blocks of any interactive or automated system. They allow a microcontroller like Arduino to perceive and respond to the physical world — detecting objects, measuring distances, or monitoring environmental conditions such as temperature and humidity. Without sensors, an Arduino would have no means of gathering information about its surroundings.

This project report covers three commonly used sensors in Arduino-based projects:

- IR (Infrared) Sensor – for object detection and proximity sensing
- Ultrasonic Sensor (HC-SR04) – for distance measurement using sound waves
- Temperature Sensor (DHT11) – for measuring ambient temperature and humidity

For each sensor, this report provides a detailed description, wiring instructions, complete Arduino code, and an explanation of how the code functions. The goal is to equip readers with both the theoretical knowledge and practical skills to integrate these sensors into their own projects.

2. IR (Infrared) Sensor

2.1 Description of the Sensor

An Infrared (IR) sensor is an electronic device that detects or emits infrared radiation in order to sense certain characteristics of its surroundings. IR sensors are widely used for object detection, obstacle avoidance, and line following in robotics.

A typical IR sensor module consists of two main components: an IR LED (transmitter) that continuously emits infrared light, and a photodiode or phototransistor (receiver) that detects reflected infrared light. When an object comes within the sensor's range, the emitted IR light bounces back and is picked up by the receiver, triggering a digital output signal.

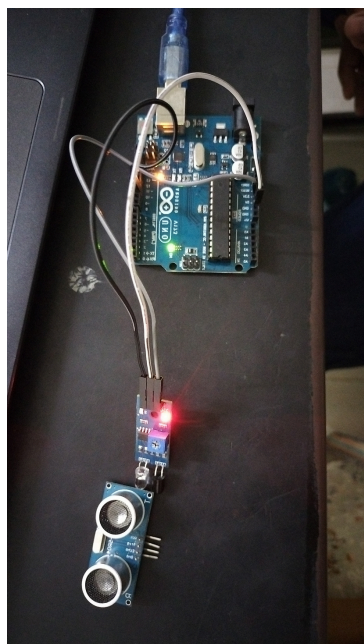
Key Specifications

- Operating Voltage: 3.3V – 5V DC
- Detection Range: 2 cm – 30 cm (adjustable via onboard potentiometer)
- Output: Digital (HIGH / LOW)
- Response Time: < 2 ms
- Common Use Cases: Obstacle avoidance robots, automatic door systems, conveyor counters

Pin Configuration

- VCC – Connect to 5V on Arduino
- GND – Connect to GND on Arduino
- OUT – Connect to a digital input pin (e.g., Pin 2)

Circuit



2.2 Arduino Code

The following sketch reads the IR sensor output and lights the built-in LED when an object is detected:

```
// IR Sensor with Arduino

// Pin Definitions

const int IR_PIN = 2;      // Digital pin connected to IR sensor output

const int LED_PIN = 13;    // Built-in LED

void setup() {

    pinMode(IR_PIN, INPUT);  // Set IR pin as input

    pinMode(LED_PIN, OUTPUT); // Set LED pin as output

    Serial.begin(9600);      // Start serial communication

    Serial.println("IR Sensor Initialized");

}

void loop() {

    int sensorValue = digitalRead(IR_PIN); // Read sensor output

    if (sensorValue == LOW) {              // Object detected (active LOW)

        digitalWrite(LED_PIN, HIGH);      // Turn LED ON

        Serial.println("Object Detected!");

    } else {                              // No object

        digitalWrite(LED_PIN, LOW);       // Turn LED OFF

        Serial.println("No Object");

    }

    delay(200); // Small delay for stability
}
```

```
}
```

2.3 How the Code Works

The sketch begins by defining two constants: IR_PIN (pin 2) connected to the sensor output, and LED_PIN (pin 13) which is the Arduino's built-in LED.

In setup(), the IR pin is configured as INPUT and the LED pin as OUTPUT. Serial communication is started at 9600 baud to display results in the Serial Monitor.

Inside loop(), digitalRead(IR_PIN) reads the sensor output. Most IR modules are active LOW — meaning the output pin goes LOW when an object is detected. When LOW is read, the LED is turned ON and "Object Detected!" is printed. Otherwise, the LED is turned OFF. A 200 ms delay stabilises the loop.

3. Ultrasonic Sensor (HC-SR04)

3.1 Description of the Sensor

The HC-SR04 is a widely used ultrasonic distance-measuring sensor that uses sound waves to calculate the distance to an object. It is inexpensive, easy to interface, and capable of measuring distances from 2 cm to 400 cm with an accuracy of approximately ± 3 mm.

The sensor works on the principle of echolocation, similar to how bats navigate. It sends a burst of ultrasonic sound pulses at 40 kHz from its transmitter. These pulses travel through the air, reflect off a surface, and return to the receiver. By measuring the time taken for this round trip, the Arduino can calculate the distance to the object.

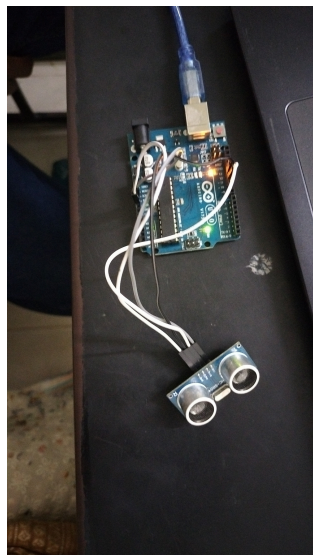
Key Specifications

- Operating Voltage: 5V DC
- Working Range: 2 cm – 400 cm
- Accuracy: ± 3 mm
- Ultrasonic Frequency: 40 kHz
- Trigger Pulse Width: 10 μ s minimum
- Common Use Cases: Distance measurement, parking assist systems, liquid level sensing

Pin Configuration

- VCC – Connect to 5V on Arduino
- GND – Connect to GND on Arduino
- TRIG – Connect to digital pin 9 (Output from Arduino)
- ECHO – Connect to digital pin 10 (Input to Arduino)

Circuit



3.2 Arduino Code

The following sketch measures distance using the HC-SR04 and prints it to the Serial Monitor:

```
// HC-SR04 Ultrasonic Sensor with Arduino

// Pin Definitions

const int TRIG_PIN = 9;    // Trigger pin

const int ECHO_PIN = 10;   // Echo pin


void setup() {

    pinMode(TRIG_PIN, OUTPUT); // Trig as output

    pinMode(ECHO_PIN, INPUT);  // Echo as input

    Serial.begin(9600);

    Serial.println("Ultrasonic Sensor Initialized");

}


void loop() {

    // Send ultrasonic pulse

    digitalWrite(TRIG_PIN, LOW);

    delayMicroseconds(2);

    digitalWrite(TRIG_PIN, HIGH);

    delayMicroseconds(10);        // 10 µs pulse

    digitalWrite(TRIG_PIN, LOW);


    // Measure echo duration

    long duration = pulseIn(ECHO_PIN, HIGH);


    // Calculate distance in cm
```

```
float distance = (duration * 0.0343) / 2.0;

// Display result

Serial.print("Distance: ");

Serial.print(distance);

Serial.println(" cm");

delay(500);

}
```

3.3 How the Code Works

The code defines TRIG_PIN (9) as an output and ECHO_PIN (10) as an input. In setup(), Serial communication is started.

In loop(), the trigger sequence begins: the TRIG pin is pulled LOW for 2 μ s to ensure a clean pulse, then set HIGH for 10 μ s to send the ultrasonic burst, and then LOW again. This is the standard HC-SR04 trigger protocol.

pulseIn(ECHO_PIN, HIGH) then waits for the echo pin to go HIGH and measures how long it stays HIGH — this duration (in microseconds) represents the round-trip time of the sound wave.

The distance is calculated using the formula: $\text{Distance} = (\text{Duration} \times \text{Speed of Sound}) / 2$. The speed of sound at room temperature is approximately 343 m/s, which equals 0.0343 cm/ μ s. Dividing by 2 accounts for the two-way journey of the pulse.

4. Temperature Sensor (DHT11)

4.1 Description of the Sensor

The DHT11 is a low-cost digital temperature and humidity sensor that uses a capacitive humidity sensor and a thermistor to measure the surrounding air. It outputs a calibrated digital signal on a single data wire, making it easy to interface with microcontrollers like the Arduino.

The DHT11 communicates using a single-wire serial protocol. The microcontroller sends a start signal, and the sensor responds with 40 bits of humidity and temperature data. A dedicated library (DHT.h) simplifies this communication and handles all the timing requirements automatically.

Key Specifications

- Operating Voltage: 3.3V – 5.5V DC
- Temperature Range: 0°C – 50°C ($\pm 2^\circ\text{C}$ accuracy)
- Humidity Range: 20% – 90% RH ($\pm 5\%$ accuracy)
- Sampling Rate: Once every 2 seconds
- Interface: Single-wire digital protocol
- Common Use Cases: Weather stations, HVAC control, greenhouse monitoring, data loggers

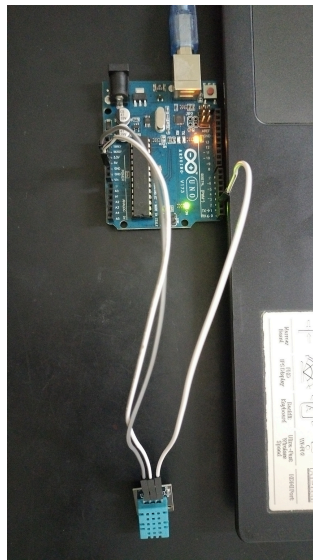
Pin Configuration

- VCC – Connect to 5V on Arduino
- GND – Connect to GND on Arduino
- DATA – Connect to digital pin 4 (with a 10k Ω pull-up resistor to VCC)

Library Required

This project requires the DHT sensor library by Adafruit. It can be installed via the Arduino IDE Library Manager: Sketch → Include Library → Manage Libraries → Search for "DHT sensor library" → Install.

Circuit



4.2 Arduino Code

The following sketch reads temperature and humidity from the DHT11 and displays the values on the Serial Monitor:

```
// DHT11 Temperature & Humidity Sensor with Arduino

#include <DHT.h>

// Pin and sensor type

#define DHTPIN 4          // Data pin connected to digital pin 4

#define DHTTYPE DHT11    // DHT 11 sensor type

DHT dht(DHTPIN, DHTTYPE);

void setup() {

    Serial.begin(9600);

    dht.begin();

    Serial.println("DHT11 Temperature Sensor Initialized");

}
```

```

void loop() {

    delay(2000); // DHT11 reads every 2 seconds


    float humidity      = dht.readHumidity();

    float tempCelsius = dht.readTemperature();

    float tempFahren    = dht.readTemperature(true); // true = Fahrenheit


    // Check for read failure

    if (isnan(humidity) || isnan(tempCelsius)) {

        Serial.println("Error: Failed to read from DHT11!");

        return;

    }


    Serial.print("Humidity: ");

    Serial.print(humidity);

    Serial.println(" %");


    Serial.print("Temperature: ");

    Serial.print(tempCelsius);

    Serial.print(" C  /  ");

    Serial.print(tempFahren);

    Serial.println(" F");

}

```

4.3 How the Code Works

The sketch begins by including the DHT.h library and defining DHTPIN (4) and DHTTYPE (DHT11). A DHT object is created with these parameters.

In `setup()`, Serial communication is initialized and `dht.begin()` starts the sensor. In `loop()`, a 2-second delay is placed at the start because the DHT11 can only provide a reading once every two seconds.

`dht.readHumidity()` and `dht.readTemperature()` fetch the current readings. Passing `true` to `readTemperature()` returns the value in Fahrenheit. The `isnan()` check validates that the readings are numbers — if the sensor is unplugged or malfunctioning, it returns NaN (Not a Number), and the sketch prints an error message instead of garbage values.

Finally, both humidity and temperature (in both Celsius and Fahrenheit) are printed to the Serial Monitor for real-time monitoring.

5. Conclusion

This project report has successfully demonstrated the working principles, circuit connections, and programming of three fundamental Arduino-compatible sensors: the IR sensor, the Ultrasonic sensor (HC-SR04), and the Temperature & Humidity sensor (DHT11). Each sensor serves a distinct purpose yet follows the same underlying workflow — detect a physical quantity, convert it into an electrical signal, and communicate that data to the Arduino for processing and display.

The IR sensor provides a fast and reliable method for binary object detection at short ranges, making it ideal for robotics and automation tasks. The Ultrasonic sensor expands this capability by providing precise distance measurements over a larger range, applicable in navigation and level sensing. The DHT11 broadens the scope of sensing to environmental monitoring, offering temperature and humidity data critical for climate control and data logging applications.

Understanding how to interface these sensors with Arduino forms the foundation of embedded systems and IoT development. The skills acquired through these experiments — reading digital/analog signals, applying mathematical formulas to raw data, using external libraries, and displaying output via Serial Monitor — are transferable to virtually any sensor integration task.

Future extensions of this project could include combining all three sensors on a single Arduino to build a smart environment monitor, integrating an LCD or OLED display for standalone data visualisation, or transmitting sensor data wirelessly using a Wi-Fi or Bluetooth module for real-time IoT applications.

— End of Report —